Phishing Analysis Using an LLM: Reflection

In this lab we ran an Ollama server in Google Colab with the quantized gemma2:9b-instruct-q4_K_M model, queried it from Python via an IP widgets interface for a single phishing analysis and then batched it against a sample email dataset.

I now understand how the stack all fits together. GPU, 4bit quantized model (a very good balance of speed and quality on the hardware we've got), the Ollama server running in the background and the Python client talking to its API. Preprocessing was just as important as the model itself. Beautiful Soup and the charset were the magic keys to getting our heads around the atrocious

HTML and encoding issues. The Pydantic schemas with instructor provided us with a very clean structured output (classification, confidence, IOCs, rationale, recommended actions) which we could then just hand straight into a ticket or a SOAR playbook.

The dataset test shifted our focus from individual verdicts to the false positives and false negatives across the whole dataset. Accuracy on its own is pretty much meaningless without that context.
Free Colab is for prototyping really, not for production. Sessions time out, you don't have guaranteed GPU access and you'll have to re-pull the model every time. The model will at times hallucinate indicators so you do need to go back and check the output against the actual email and headers.

In the SOC that would be Tier 1 decision support on the phishing queue that's been reported. The analyst would be the one making the final call, verifying the IOCs prior to blocklisting and approving the response actions. Emails can also contain prompt injection attempts, which is another reason to have a human in the loop.

Next, we'd run a read only pilot with the analysts for a few weeks, comparing our judgements against theirs to establish precision and recall before any auto-enrichment at all, starting with governance on how the email is stored and logged.